15-using-test-doubles-effectively/06/subscription_service/lib/recurring_payment.rb
```
class RecurringPayment
  def self.process_subscriptions(subscriptions)
    subscriptions.each do |subscription|
      CashCow.charge_card(subscription.credit_card, subscription.amount)
      # ...send receipt and other stuff...
    end
  end
end
```

The unit spec for this class verifies that we're charging the correct amounts:

15-using-test-doubles-effectively/07/subscription_service/spec/recurring_payment_spec.rb
```
RSpec.describe RecurringPayment do
  it 'charges the credit card for each subscription' do
    card_1 = Card.new(:visa, '1234 5678 9012 3456')
    card_2 = Card.new(:mastercard, '9876 5432 1098 7654')

    subscriptions = [
      Subscription.new('John Doe', card_1, 19.99),
      Subscription.new('Jane Doe', card_2, 29.99)
    ]

    expect(CashCow).to receive(:charge_card).with(card_1, 19.99)
    expect(CashCow).to receive(:charge_card).with(card_2, 29.99)

    RecurringPayment.process_subscriptions(subscriptions)
  end
end
```

Here, we are using CashCow as a mock object. We expect it to receive the :charge_card message for each subscription. Crucially, this message expectation also serves to stub the method, preventing a real charge from going through in our tests.

Hundreds of customers later, we may find that our RecurringPayment class is spending a lot of time making a separate API call for each subscription. In this case, we can switch to CashCow's *bulk* interface, which lets us make a single API call to charge *all* of our customers' cards.

Here's an updated RecurringPayment class that uses the bulk API call:

15-using-test-doubles-effectively/08/subscription_service/lib/recurring_payment.rb
```
class RecurringPayment
  def self.process_subscriptions(subscriptions)
    cards_and_amounts = subscriptions.each_with_object({}) do |sub, data|
      data[sub.credit_card] = sub.amount
    end

    CashCow.bulk_charge_cards(cards_and_amounts)
    # ...send receipts and other stuff...
  end
end
```